॥ श्री गणेशाय नमः ॥

Artificial Intelligence

Search

State: Representation of physical configuration.
Node: Includes state, parent node, action, path cost $g(x)$, depth.

Time complexity:
$b \rightarrow$ max branching factor, $d \rightarrow$ depth of least cost soln, $m \rightarrow$ max depth of state space

optimal: always finds least cost soln, systematicity: visits each state atmost once.

uniformed search techniques:

Depth-first search:
→ maintain stack
Time: $O(b^m)$ Space: $O(bm)$
complete ✗
optimal ✗

Breadth-first search:
→ maintain queue
Time: $O(b^d)$ Space: $O(b^d)$
complete ✓
optimal ✓ if step-cost = 1

Depth limited search:
→ Limit DFS to a finite # of levels.
Time: $O(b^l)$ space: $O(bl)$
complete ✗
optimal ✗

Iterative deepening search:
→ Apply depth limited search at increasing levels of depth.
Time: $O(b^d)$ space: $O(bd)$
complete ✓ optimal ✓ If step cost = 1

Uniform cost Search (UCS):
→ Dijkstra's. Choose cheapest node first.
→ Priority queue is used.
→ $f(n) = g(n)$
Time: $O(b^d) = O(b^{1+\lfloor \frac{c^*}{e} \rfloor})$ Space: $O(b^d) = O(b^{1+\lfloor \frac{c^*}{e} \rfloor})$
$c^* \rightarrow$ total optimal cost
$e \rightarrow$ minimum cost edge
complete ✓ (no -ve edges, b is finite) Optimal ✓ (no -ve cycles)

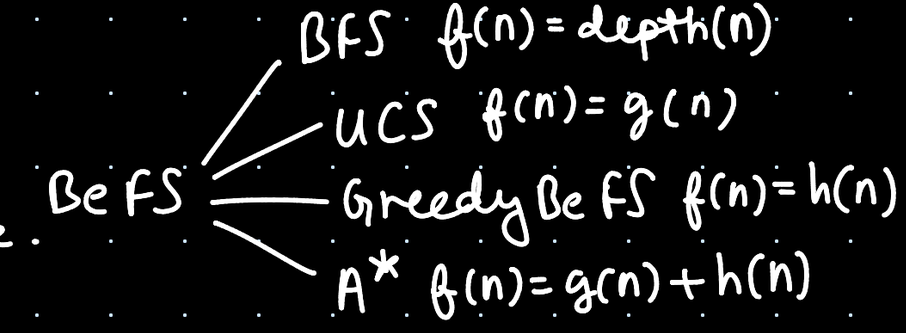
Bidirectional search:
→ start search at both ends: start and goal. Hope they meet.
Time: $O(b^{d/2} + b^{d/2}) = O(b^{d/2})$ Space: $O(b^{d/2})$
complete ✓ optimal ✓ (BFS)

Beam Search:
→ keep a fixed size frontier. Prune worst nodes if they exceed.
complete ✗ optimal ✗

Informed Search techniques:

- use an evaluation function to choose which node to expand.
- Priority queue is used.

Best-first search:
- Expand unvisited node with the lowest f(n) value.

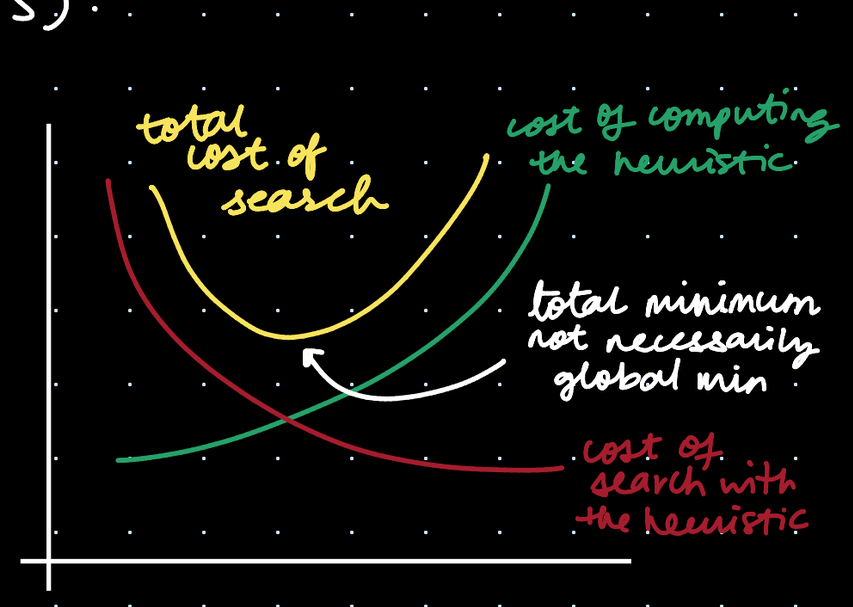


Greedy BeFS:
- f(n) = h(n)
- Time: $O(b^m)$ space: $O(b^m)$
  complete ✗ optimal: ✗

A* search:
- f(n) = g(n) + h(n)
- Time: $O(b^d)$ space: $O(b^d)$ Ideally $b^* = 0$ as h( ) should prune branches.
  complete ✓ optimal ✓ (with an admissible heuristic)

Iterative deepening A*:
- DO DFS but prune when cost(f) > current bound. Increase bound.

weighted A*:
- f(n) = g(n) + w·h(n) where w > 1 (generally 5).
  - w=1 ⇒ regular A*
    w=0 ⇒ UCS
    w=∞ ⇒ greedy BeFS
- Admissibility may NOT hold since we change h → wh.
- optimal ✗

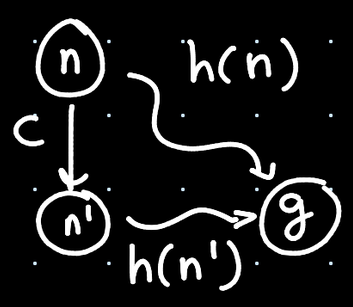


Relaxed Problem:
- Problem with fewer restrictions on the actions.
- cost of relaxed problem is an admissible heuristic to the original problem.

Heuristics

- Heuristic function $h(n)$ calculates estimated cost from node n to goal.

Admissible heuristic:
- Never overestimates. $h(n) \leq h^*(n)$

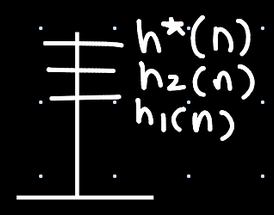


consistent heuristic:
- $\forall n$, $\forall$ successors n', $h(n) \leq c + h(n')$

consistency implies admissibility.

- more informed heuristic means less search.

Dominating heuristic:
h_2 dominates $h_1 \equiv h_2(n) \geq h_1(n) \quad \forall n$ (both admissible)
- h_2 is better for search.
(more informed)



Common heuristics:

Manhattan distance → for 8-puzzle (dist from start to goal). For path problems.
↳ Admissible.

# of misplaced tiles → for 8 puzzle
↳ Admissible.

Euclidean distance → for path problems.
↳ Admissible.

- If h_1 and h_2 are admissible:
  - → $\min(h_1, h_2)$
  - → $\max(h_1, h_2)$
  - → $(h_1 + h_2)/2$
  - → $(h_1 h_2)^{0.5}$
  - → $|h_1 - h_2|$

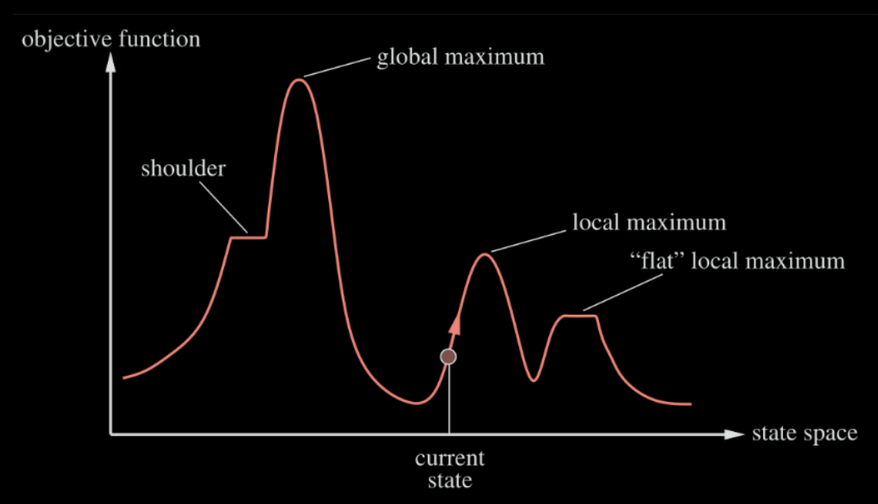
  all are admissible (just check that they don't overestimate given h_1 and h_2 don't)

- h_1 & h_2 are admissible.
  $\max(h_1, h_2)$ dominates both.

- Say h is admissible. (h+1) is inadmissible.

Local Search

- Search from a start state to neighboring states without keeping track of path.
- usually optimizaⁿ problems. can be converted to CSPs.
- less memory requirements. Sub-optimal but okayish solution.
- Drawbacks:
  - → local maxima
  - → plateau
  - → shoulder → sideways move to escape
  - → greedy nature



Tabu Search:
- Keep last k visited states in a tabu list.
- Never make the step that is currently Tabu'ed.
- A reasonably (100 or so) sized Tabu list keeps the search non-redundant.

Escaping shoulders: Perform BFS from local optima to find next state with better h function value.

Stochastic Hill climbing

Random walk hill climbing:
- At each step, do greedy or random step.

↙ neighbor with prob. p ↘ jump with prob (1-p)

Hill climbing with random restarts:
- For each restart, run until termination or for a fixed time.

Hill climbing with both:
- At each step, choose greedy, random, or restart step.

Simulated annealing:
- Change in objective funcⁿ = δ for next move.
- $\delta > 0$ → make that move. else make that move with prob δ.

Local beam search:
- start with k nodes. Keep track of k nodes rather than a single node.
- If no info is shared → same as k parallel searches.

Stochastic beam search with k states.

Adversarial Search

△ → max node : tries to maximize ; initialize to $-\infty$; ▽ → min node : tries to minimize ; initialize to $+\infty$;

Time : $O(b^m)$ space : $O(b^m)$

complete ✓ optimal ✓ (If P2 plays optimally)

Alpha-beta pruning :

α : lower bound on the value that a max node may finally be assigned.

$V \geq \alpha$ highest (best) value for MAX along the path.

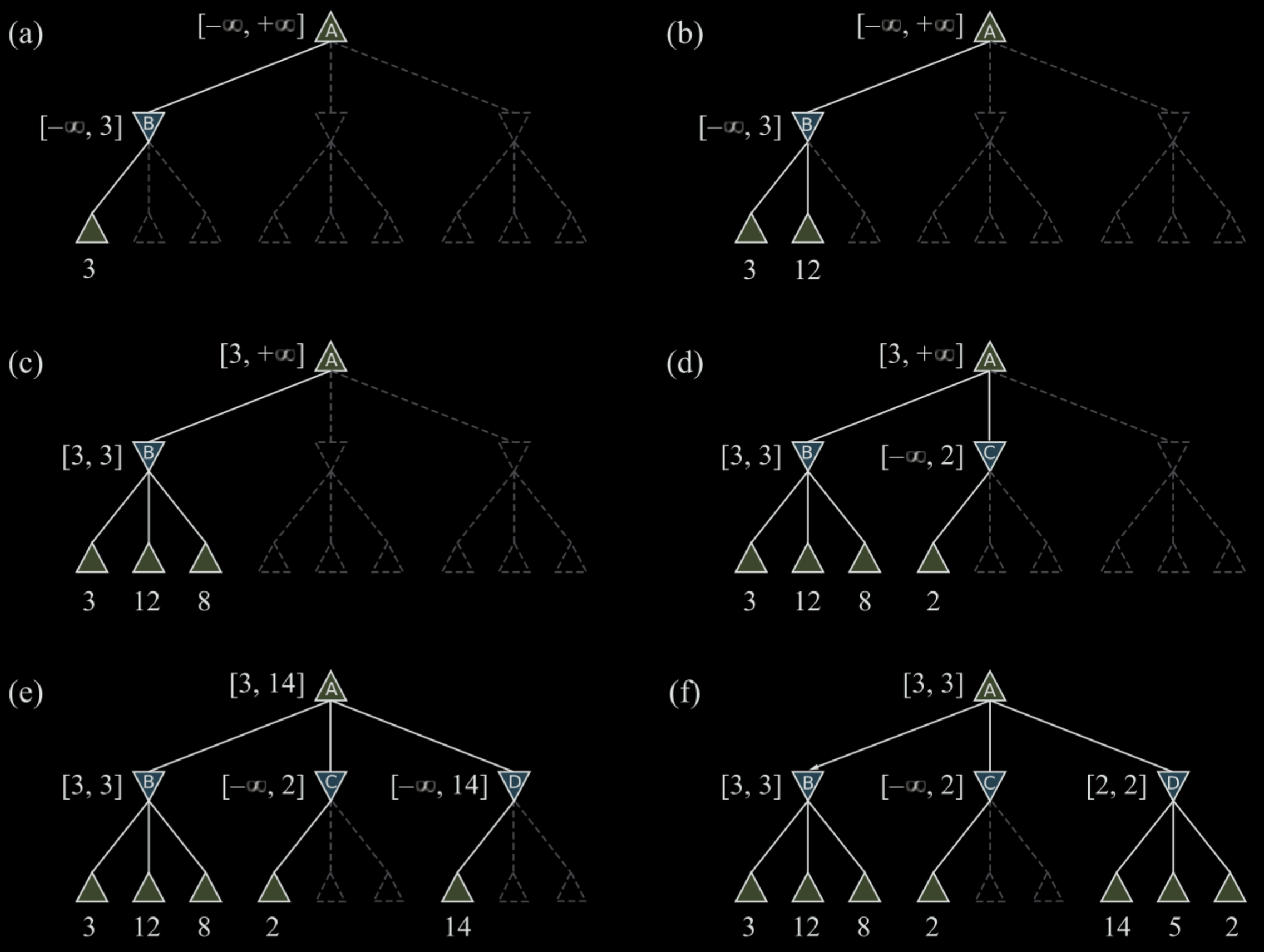
β : upper bound on the value that a min node may finally be assigned.

$V \leq \beta$ lowest (best) value for MIN along the path.

Pruning condition : $\alpha \geq \beta$

Time : worst → $O(b^d)$

Best → $O(\sqrt{b^d})$



Logic

Precedence of logical operators: $\sim$ $\wedge$ $\vee$ $\rightarrow$ $\leftrightarrow$

Laws:

$P \leftrightarrow Q \equiv PQ + \bar{P}\bar{Q} \equiv (P \rightarrow Q) \wedge (Q \rightarrow P)$

$(P \rightarrow R) \vee (Q \rightarrow R) \equiv (P \wedge Q) \rightarrow R$

Inference:

consistent systems: no contradictions between statements.

Arguments: $(P_1 \wedge P_2 \wedge \dots \wedge P_n) \rightarrow Q$ is true $\Rightarrow$ $\dfrac{\begin{array}{l}P_1\\P_2\\\vdots\\P_n\end{array}}{Q}$ is a valid argument.

Valid arguments:
- sound: all premises are valid, not just some.
- unsound (even $P=F$ and $Q=T$ makes $P \rightarrow Q$ true).

Rules of inference:
- Modus ponens: $P \rightarrow Q, P \vDash Q$
- modus tollens: $P \rightarrow Q, \sim Q \vDash \sim P$
- Hypothetical syllogism: $P \rightarrow Q, Q \rightarrow R \vDash P \rightarrow R$
- Disjunctive syllogism: $P \vee Q, \sim P \vDash Q$
- Conjunctive syllogism: $P \wedge Q, P \vDash Q$
- Addition: $P \vDash P \vee Q$
- Simplification: $P \wedge Q \vDash P$
- Dilemma: $P \vee Q, P \rightarrow R, Q \rightarrow R \vDash R$
- Constructive dilemma: $P \vee Q, P \rightarrow R, Q \rightarrow S \vDash R \vee S$
- Destructive dilemma: $P \rightarrow R, Q \rightarrow S, \sim R \rightarrow \sim S \vDash \sim P \vee \sim Q$ → same as $P \rightarrow \sim Q$

Fallacies

- Affirming the consequence: $P \rightarrow Q, Q \vDash P$
- Denying the antecedent: $P \rightarrow Q, \sim P \vDash \sim Q$
- Non sequitur: $P, Q \vDash R$
  (It does not follow.)

Dual:

AND → OR
OR → AND
T → F
F → T

} make these changes to get the dual of an expression

Quantifiers: convert predicates to propositions.

$\forall$, $\exists$, $\exists!$ or $\exists_{=1}$

- $\forall$: universal
- $\exists$: existential quantification
- $\exists!$: uniqueness

variable: free / bounded

Horn clause:
- A disjunction of literals with atmost 1 positive literal. (e.g. $\neg p \vee \neg q \vee r \vee \neg s$) $\rightarrow$ +ve

Definite HC: exactly 1 positive literal.
Negative HC: 0 +ve literals.

Chaining — Forward / Backward

Knowledge base and formula:
- KB entails f: $KB \models f$
- KB contradicts f: $KB \models \neg f$
- KB contingent to f: f doesn't contradict KB. It adds some non-trivial info.

To convert to CNF:
- convert to truth table.
- for every $\phi = 0$, get a clause by putting literal if it takes 0, its not if it takes 1.

a	b	q	r	z	ϕ
0	0	0	0	0	0
0	0	0	0	1	1
0	0	0	1	0	0

$(a \vee b \vee q \vee \bar{r} \vee \bar{z}) \wedge (a \vee b \vee q \vee \bar{r} \vee z) \wedge (a \vee b \vee q \vee r \vee z)$

Reasoning under uncertainty

Marginal prob: $P(X=x) = \int_{-\infty}^{\infty} f_{x,y}(x,y)\,dy$ or $\sum_y P(X=x, Y=y)$

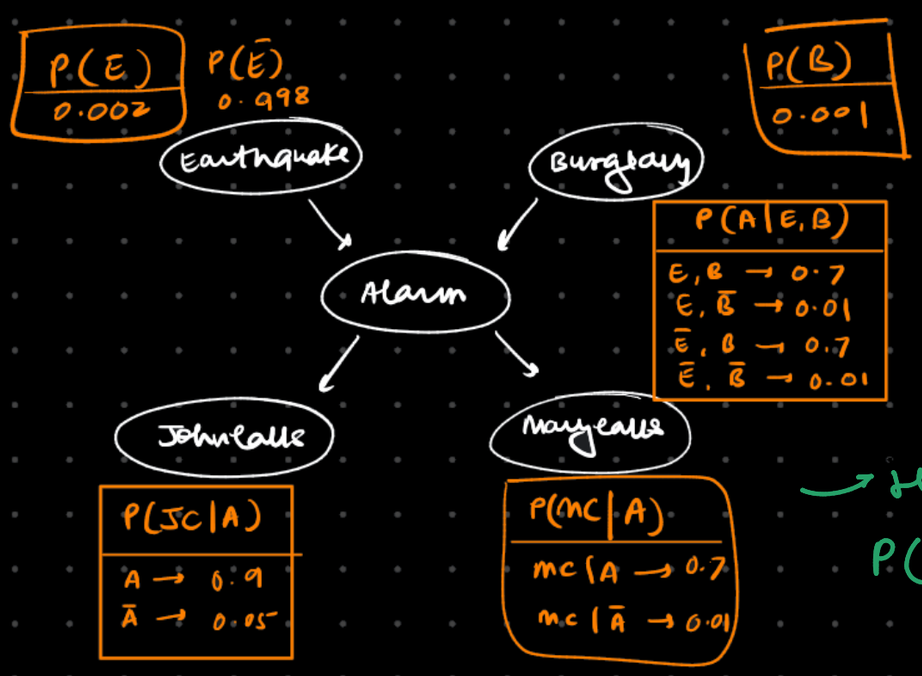
Bayes Theorem = $P(\text{cause}|\text{effect}) = \frac{P(\text{effect}|\text{cause}) \cdot P(\text{cause})}{P(\text{effect})}$

Naive Bayes assumption: All features are conditionally independent.

$$P(\text{cause}|\text{effect}_1, \text{effect}_2, \ldots, \text{effect}_n) = P(\text{cause}) \cdot \prod_i P(\text{effect}_i|\text{cause})$$

Bayesian networks (belief networks)

- DAG: node is random variable, edges denote direct probabilistic inference.



Theorem of factorization:

Full joint distribution is given as:

$$P(x_1, x_2, \ldots, x_n) = \prod_{i=1}^{n} P(x_i|\text{Par}(x_i))$$

→ Here:

$$P(J, M, A, \sim B, \sim E) = P(J|A) \cdot P(M|A) \cdot P(A|\sim B, \sim E) \cdot P(\sim B) \cdot P(\sim E)$$

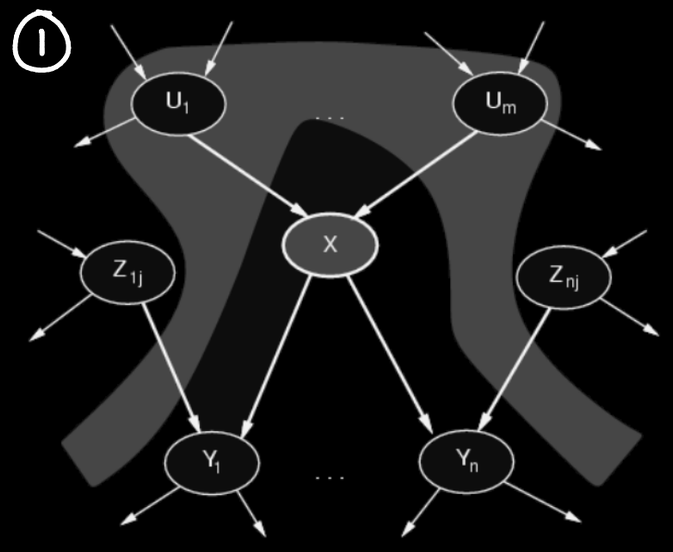
To calculate the full joint distribution, # of distinct probabilities read: $2^{\#\text{ of parents}}$

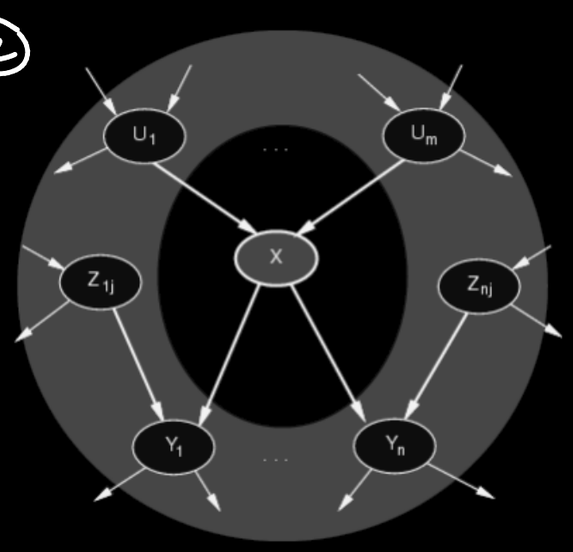
Conditional independence:

Rule #1: Given the parents, X is conditionally independent of non-descendants.

Rule #2: Given the markov blanket, X is independent of all other nodes.

Markov Blanket(X) = Parent(X) ∪ children(X) ∪ parent(children(X))

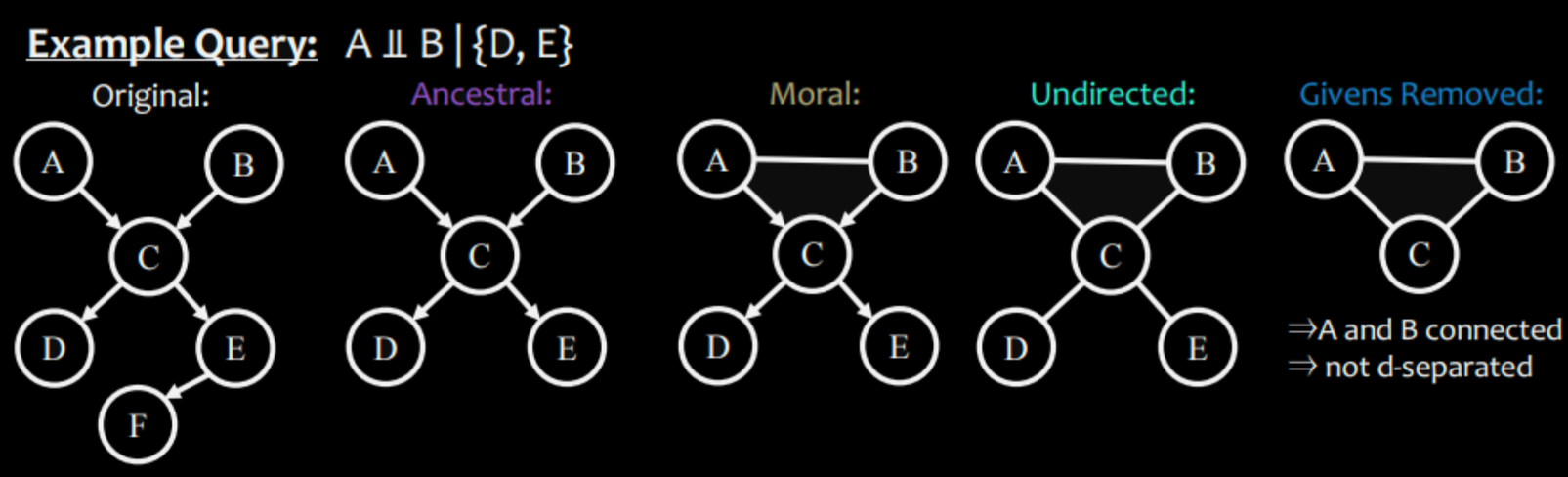
① 

② 

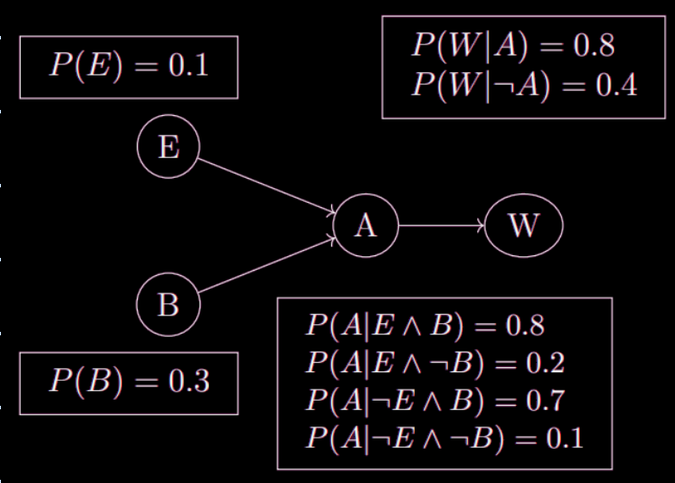
d-separation: To check if X & Z are d-separated given set E.

① Draw ancestral graph of X, Z, E.
② Moralize each node's parents.
③ Disorient: directed edges → undirected.
④ Delete givens (E).

If X and Y have multiple variables, check all pairs of X_i and Y_j:



Inference
- exact (variable elimination)
- approximate (sampling)



Variable elimination:

Find $P(B|\sim A)$

$$P(B, \sim A) = \sum_e \sum_w P(B) \cdot P(e) \cdot P(\sim a|B, e) \cdot P(w|\sim a)$$

$$P(B, \sim A) = P(B) \cdot \sum_e P(e) \cdot P(\sim a|B, e) \cdot \sum_w P(w|\sim a)$$

step 1: define factors

$f_1(B)$:

B	val
T	0.3
F	0.7

$f_2(E)$:

E	val
T	0.1
F	0.9

$f_3(A, B, E)$:

A	B	E	val
T	T	T	0.8
T	T	F	0.7
T	F	T	0.2
T	F	F	0.1
F	T	T	0.2
F	T	F	0.3
F	F	T	0.8
F	F	F	0.9

$f_4(W, A)$:

W	A	val
T	T	0.8
T	F	0.4
F	T	0.2
F	F	0.6

step 2: restrict to given values ($\sim a$)

$f_5(B, E)$

B	E	val
T	T	0.2
T	F	0.3
F	T	0.8
F	F	0.9

$f_6(W)$:

W	val
T	0.4
F	0.6

now factors are $f_1(B)$, $f_2(E)$, $f_5(B, E)$, $f_6(W)$

$$P(B, \sim A) = P(B) \cdot \sum_e P(e) \cdot P(\sim a|B, e) \cdot \sum_w P(w|\sim a)$$

step 3: now do the operations to eliminate variables.

$\sum_w P(w|\sim a)$

W	val
T	0.4
F	0.6

$f_7()$:

val
1.0

← put $\sum_w P(w|\sim a) = 1$

$$P(B, \sim A) = P(B) \cdot \sum_e P(e) \cdot P(\sim a|B, e)$$

multiply

$f_2(E)$:

E	val
T	0.1
F	0.9

$\cdot$ $f_5(B, E)$

B	E	val
T	T	0.2
T	F	0.3
F	T	0.8
F	F	0.9

$= f_8(B, E)$:

B	E	val
T	T	0.02
T	F	0.27
F	T	0.08
F	F	0.81

now take summation over e:

B	val
T	0.29
F	0.89

← $f_9(B)$

$$P(B, \sim A) = P(B) \cdot f_9(B)$$

B	val
T	0.3
F	0.7

$\cdot$

B	val
T	0.29
F	0.89

$= f_{10}(B)$:

B	val
T	0.087
F	0.623

step 4: now normalize

$f_{11}(B)$:

B	val
T	0.1225
F	0.8775

$$P(b|\sim a) = 0.1225$$

Sampling

Prior sampling:

- Generate samples without any conditions or evidence.
- working:
  - start at roots and sample based on prior prob.
  - move through the network (par → children). sample based on conditional prob. on parents' sample values.
  - repeat till you have the read. # of samples.

The generated samples can be used to estimate marginal probabilities by counting.

$P(C)$

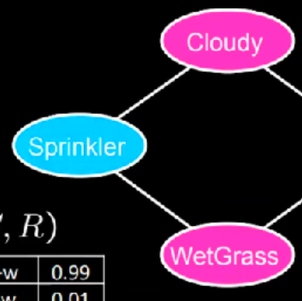
+c	0.5
-c	0.5

$P(S|C)$

+c	+s	0.1
	-s	0.9
-c	+s	0.5
	-s	0.5

$P(R|C)$

+c	+r	0.8
	-r	0.2
-c	+r	0.2
	-r	0.8



$P(W|S,R)$

+s	+r	+w	0.99
		-w	0.01
	-r	+w	0.90
		-w	0.10
-s	+r	+w	0.90
		-w	0.10
	-r	+w	0.01
		-w	0.99

Samples:
+c, -s, +r, +w
+c, +s, +r, +w
-c, +s, +r, -w
+c, -s, +r, +w
-c, -s, -r, +w

use given samples

$P(+c) = \frac{3}{5}$

$P(-s) = 3/5$ $\quad P(-s, r) = 2/5$

Rejection sampling

- Prior sampling, but some evidence.
- Reject samples inconsistent with evidence.
- Problem:
  If evidence is unlikely, you reject a lot of samples.

Samples:
+c, -s, +r, +w ✗
+c, +s, +r, +w
-c, +s, +r, -w
+c, -s, +r, +w ✗
-c, -s, -r, +w ✗

$P(+c|+s,+r) = 1/2$

Likelihood Weighting:

- Fix evidence variables and sample the rest.
- As sampling distribution is inconsistent, weight by probability of evidence given parents.

$P(C)$

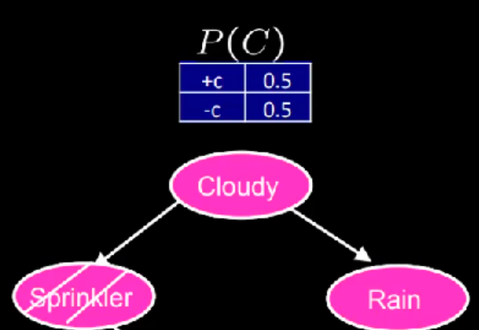
+c	0.5
-c	0.5

$P(S|C)$

+c	+s	0.1
	-s	0.9
-c	+s	0.5
	-s	0.5

$P(R|C)$

+c	+r	0.8
	-r	0.2
-c	+r	0.2
	-r	0.8



$P(W|S,R)$

+s	+r	+w	0.99
		-w	0.01
	-r	+w	0.90
		-w	0.10
-s	+r	+w	0.90
		-w	0.10
	-r	+w	0.01
		-w	0.99

Samples:
+c, +s, +r, +w
...

$w = 1.0 \times 0.1 \times 0.99$

$P(+c|+s,+w)$ ← calc. prob. of evidence given parents.

$P(+s|+c) \cdot P(+w|+s,+r) \times 1$
$= (0.1)(0.99)$ ← weight of particular sample.

weight of particular sample = prob. of evidence given parents (in sample).

To find $P(+c|-s,+w)$

For w_1: Joint prob is
$P(+c) \cdot P(-s|+c) \cdot P(+r|+c) \cdot P(+w|-s,+r)$
P(evidence | parents) is considered
$P(-s|+c) \cdot P(+w|-s,+r) = (0.9)(0.9) = 0.81$

$w_2 = P(-s|-c) \cdot P(+w|-s,-r) = (0.5)(0.01) = 0.005$

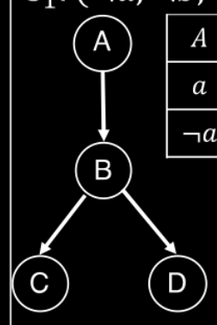
Samples:
+c, -s, +r, +w → $w_1 = 0.81$
+c, +s, +r, +w ✗
-c, +s, +r, -w ✗
+c, -s, +r, +w → 0.81
-c, -s, -r, +w → $w_2 = 0.005$

$P(+c|-s,+w) = \frac{0.81}{0.81 + 0.005} = 0.9938$

Q.32

Consider the Bayes Net containing four Boolean random variables (A, B, C, D), with the following convention: $A = True \Rightarrow A = a, and\ A = False \Rightarrow A = \neg a$; and similarly for the other variables. The conditional probability tables for the nodes in the network are also indicated in the figure. The following samples were generated through likelihood weighting:

$s_1: (\neg a, \neg b, \neg c, \neg d)$; $s_2: (\neg a, b, \neg c, \neg d)$; $s_3: (\neg a, \neg b, \neg c, d)$; $s_4: (\neg a, b, \neg c, d)$



A	$P(A)$
a	1/5
$\neg a$	4/5

A	B	$P(B\|A)$
a	b	1/5
$\neg a$	b	1/2
a	$\neg b$	4/5
$\neg a$	$\neg b$	1/2

B	C	$P(C\|B)$
b	c	1/4
$\neg b$	c	3/4
b	$\neg c$	2/5
$\neg b$	$\neg c$	3/5

B	D	$P(D\|B)$
b	d	1/2
$\neg b$	d	4/5
b	$\neg d$	1/2
$\neg b$	$\neg d$	1/5

Estimate the likelihood weight of each sample and thereby estimate $P(b|\neg a, \neg c)$

(A)	$s_1: 0.48, s_2: 0.32, s_3: 0.48, s_4: 0.32, P(b\|\neg a, \neg c) = 0.4$
(B)	$s_1: 0.48, s_2: 0.32, s_3: 0.48, s_4: 0.32, P(b\|\neg a, \neg c) = 0.64$
(C)	$s_1: 0.32, s_2: 0.48, s_3: 0.48, s_4: 0.32, P(b\|\neg a, \neg c) = 0.64$
(D)	$s_1: 0.48, s_2: 0.32, s_3: 0.32, s_4: 0.32, P(b\|\neg a, \neg c) = 0.4$

→ $P(b|\neg a, \neg c)$

$s_1(\neg a, \neg b, \neg c, \neg d) \rightarrow w_1 = P(\neg a) \cdot P(\neg c|\neg b) = \frac{4}{5} \cdot \frac{3}{5} = \frac{12}{25} = 0.48$

$s_2(\neg a, b, \neg c, \neg d) \rightarrow w_2 = P(\neg a) \cdot P(\neg c|b) = \frac{4}{5} \cdot \frac{2}{5} = \frac{8}{25} = 0.32$

$s_3(\neg a, \neg b, \neg c, d) \rightarrow w_3 = P(\neg a) \cdot P(\neg c|\neg b) = 0.48$

$s_4(\neg a, b, \neg c, d) \rightarrow w_4 = P(\neg a) \cdot P(\neg c|b) = 0.32$

$$P(b|\neg a, \neg c) = \frac{0.32 + 0.32}{0.48 + 0.32 + 0.48 + 0.32} = 0.4$$

Markov chain monte carlo

markov chain : model that describes a sequence of random events where the prob. of transitioning to the next state depends only on the current one. (markov property / memorylessness).

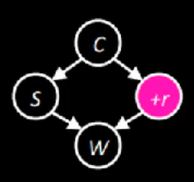
$$P(X_{t+1} = S_{t+1} | S_t, S_{t-1}, \cdots, S_0) = P(X_{t+1} = S_{t+1} | S_t)$$

monte carlo Sampling :
- repeated random sampling to obtain numerical results.
- use randomness to solve otherwise deterministic problems.
- Generate inputs, perform deterministic computation of outputs, aggregate results.

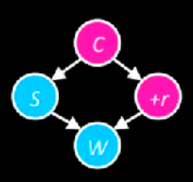
Gibbs Sampling :
- Iteratively sample each variable X_i conditional on all other variables.
- These conditions distributions are easier to sample, compared to full joint distributions.

Gibbs Sampling Example P(S|+r)

- Step 1: Fix evidence
  - R = +r



- Step 2: Initialize other variables
  - Randomly



- Steps 3: Repeat
  - Choose a non-evidence variable X
  - Resample X from P(X | all other variables)

Sample from $P(S|+c,-w,+r)$ Sample from $P(C|+s,-w,+r)$ Sample from $P(W|+s,+c,+r)$

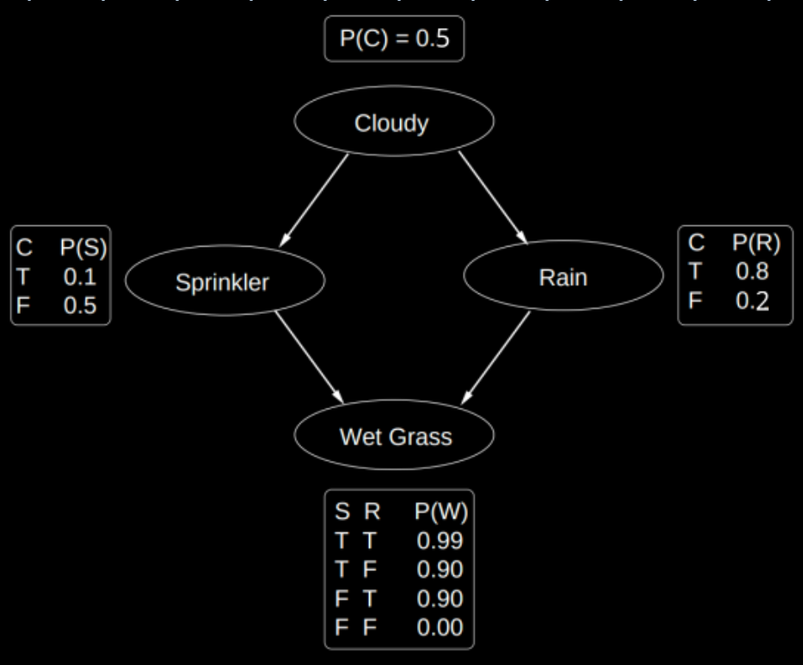
another quesⁿ : $P(c|s=t, r=f)$

step 1 : initialize

to calculate $P(c|s=t, r=f)$ ← c: query, s, r: givens

step 2 : calculate $P(c|S=T, R=F, W=F)$ ← c: all remaining nodes

$= P(c|MB(C))$ ← markov blanket



$$P(x_i | mb(x_i)) = \alpha \cdot P(x_i | Parents(x_i)) \cdot \prod_{j \in children(x_i)} P(y_j | Parents(y_j))$$

$$P(c | mb(c)) = \alpha \cdot P(c) \cdot P(s | Parents(s)) \cdot P(R | Parents(R))$$

$$= \alpha \cdot P(c) \cdot P(s|c) \cdot P(R|c)$$

$P(C=T | S=t, R=f) = \alpha \cdot (0.5)(0.1)(0.2) = 0.01\alpha$

$P(C=F | S=t, R=f) = \alpha \cdot (1-0.5)(0.5)(1-0.2) = 0.2\alpha$

C=T	C=F
$\alpha \langle 0.01, 0.2 \rangle$	
↓ normalize	
$\alpha \langle 0.0476, 0.9524 \rangle$	